



UNIVERSIDAD PERUANA
CAYETANO HEREDIA

CICORINA: Sistema de Detección de Estrés Cognitivo basado en Variabilidad de Frecuencia Cardíaca (HRV) para el Monitoreo en Estudiantes Universitarios

Código Fuente

Julio 2026

CÓDIGO PYTHON

```
import streamlit as st
from pathlib import Path
import time
import datetime

import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import butter, filtfilt, iirnotch, find_peaks, welch
from scipy.interpolate import interp1d
from scipy.integrate import trapezoid

# -----
# Inicialización de Session State
# -----
if "historial" not in st.session_state:
    st.session_state.historial = [] # Lista de diccionarios con resultados
if "registro_actual" not in st.session_state:
    st.session_state.registro_actual = None # Almacena el último resultado procesado
if "vista_detalle_id" not in st.session_state:
    st.session_state.vista_detalle_id = None # ID del registro que se está viendo en el historial
if "form_version" not in st.session_state:
    st.session_state.form_version = 0

if "autenticado" not in st.session_state:
    st.session_state.autenticado = False
if "usuario_actual" not in st.session_state:
    st.session_state.usuario_actual = None

# Base de datos simulada de usuarios (Usuario: Contraseña)
if "usuarios_db" not in st.session_state:
    st.session_state.usuarios_db = {
        "admin": "1234",
        "U20261234": "password123"
    }

# Control para alternar entre la pantalla de Login y la de Registro
if "pantalla_actual" not in st.session_state:
    st.session_state.pantalla_actual = "login"

# -----
# Función: Mostrar Resultados
# -----
def mostrar_bloque_resultados(datos):
    """Función para no repetir código al mostrar resultados"""
    st.subheader(f" Análisis del {datos['fecha']}")
```

```

st.markdown("### 📄 Evaluación Subjetiva")
col_sub1, col_sub2, col_sub3, col_sub4 = st.columns(4)
with col_sub1:
    st.metric("Puntaje PSS-14", datos['pss'])
with col_sub2:
    st.metric("Clasificación PSS-14", datos['clasificacion'])
with col_sub3:
    st.metric("Horas de estudio/semana", datos['horas'])
with col_sub4:
    st.metric("Género", datos.get('genero', 'No especificado'))

```

```

st.divider()

```

```

st.markdown("### ❤️ Parámetros Cardíacos y HRV (Basal)")
col_m1, col_m2 = st.columns(2)
with col_m1:
    st.metric("Frecuencia Cardíaca Media", f"{datos['mean_hr']:.2f} BPM")
    st.metric("SDNN (Variabilidad Temporal)", f"{datos['sdnn']:.2f} ms")
with col_m2:
    st.metric("RMSSD (Actividad Parasimpática)", f"{datos['rmssd']:.2f} ms")
    st.metric("Relación LF/HF (Balance Simp/Parasimp)", f"{datos['lf_hf_ratio']:.2f}")

```

```

st.divider()
st.subheader("📊 Gráficos de Análisis de Señal")
st.pyplot(datos['figura'])

```

```

# =====
# LÓGICA DE CONTROL DE ACCESO
# =====
if not st.session_state.autenticado:
    st.markdown("<h1 style='text-align: center;*>🌿 Sistema CICORINA</h1*>",
unsafe_allow_html=True)
    st.markdown("<p style='text-align: center; color: gray;*>Sistema Inteligente para la Clasificación
del Nivel de Estrés Universitario</p*>", unsafe_allow_html=True)

```

```

st.divider()

```

```

with st.container(border=True):
    st.markdown("### 🌈 ¡Bienvenido al Asistente CICORINA!")
    st.write(
        """
        Este sistema combina la evaluación de tu percepción psicológica de estrés (**Cuestionario
PSS-14**)
        junto con el análisis automatizado de tus señales cardíacas reales (**ECG - BITalino**) para
ofrecerte
        un diagnóstico objetivo de tu estado de salud y reactividad ante el estrés agudo.
        """
    )

```

```
)
    st.info("💡 Para comenzar a procesar tus señales y guardar tu historial, por favor inicia sesión o crea una cuenta nueva abajo.")
```

```
st.write("")
```

```
opcion_acceso = st.radio(
    "Selecciona una opción para continuar:",
    ["🔑 Iniciar Sesión", "📝 Crear Cuenta Nueva (Sign Up)"],
    horizontal=True,
    label_visibility="collapsed"
)
```

```
st.write("")
```

```
# -----
```

```
# PANTALLA DE INICIO DE SESIÓN (LOGIN)
```

```
# -----
```

```
if "🔑 Iniciar Sesión" in opcion_acceso:
```

```
    with st.form("formulario_login_bonito"):
```

```
        st.markdown("#### 🔒 Ingreso al Sistema")
```

```
        usuario = st.text_input("Identificación / Código de Estudiante", placeholder="Ej: U20261234")
```

```
        contrasena = st.text_input("Contraseña", type="password", placeholder="*****")
```

```
st.write("")
```

```
boton_ingresar = st.form_submit_button("🚀 Ingresar al Sistema", use_container_width=True)
```

```
if boton_ingresar:
```

```
    if usuario == "" or contrasena == "":
```

```
        st.warning("⚠️ Por favor complete todos los campos.")
```

```
    elif usuario in st.session_state.usuarios_db and st.session_state.usuarios_db[usuario] ==
```

```
contrasena:
```

```
        st.session_state.autenticado = True
```

```
        st.session_state.usuario_actual = usuario
```

```
        st.success("¡Ingreso exitoso! Cargando entorno...")
```

```
        time.sleep(1)
```

```
        st.rerun()
```

```
    else:
```

```
        st.error("❌ Identificación o contraseña incorrecta. Intente de nuevo.")
```

```
# -----
```

```
# PANTALLA DE REGISTRO (SIGN UP)
```

```
# -----
```

```
else:
```

```
    with st.form("formulario_registro_bonito"):
```

```
        st.markdown("#### 📝 Registro de Nuevo Usuario")
```

```
        nuevo_usuario = st.text_input("Defina su Identificación / Código", placeholder="Ej: U20265555")
```

```

nueva_contrasena = st.text_input("Defina su Contraseña", type="password",
placeholder="Mínimo 4 caracteres")
confirmar_contrasena = st.text_input("Confirme su Contraseña", type="password",
placeholder="*****")

st.write("")
boton_registrar = st.form_submit_button("🎉 Registrar Cuenta", use_container_width=True)

if boton_registrar:
    if nuevo_usuario == "" or nueva_contrasena == "" or confirmar_contrasena == "":
        st.warning("⚠️ Todos los campos son obligatorios.")
    elif nuevo_usuario in st.session_state.usuarios_db:
        st.error("❌ El usuario ya se encuentra registrado.")
    elif nueva_contrasena != confirmar_contrasena:
        st.error("❌ Las contraseñas no coinciden.")
    elif len(nueva_contrasena) < 4:
        st.error("❌ La contraseña debe tener al menos 4 caracteres.")
    else:
        st.session_state.usuarios_db[nuevo_usuario] = nueva_contrasena
        st.success("🎉 ¡Cuenta creada con éxito! Selecciona 'Iniciar Sesión' arriba para
ingresar.")
        time.sleep(1.5)
else:
# =====
# ESTE BLOQUE SOLO SE EJECUTA SI EL USUARIO YA ESTÁ COMPROBADO (AUTENTICADO)
# =====

with st.sidebar:
    st.write(f"👤 **Usuario:** `{st.session_state.usuario_actual}`")
    if st.button("🔒 Cerrar Sesión", use_container_width=True):
        st.session_state.autenticado = False
        st.session_state.usuario_actual = None
        st.session_state.registro_actual = None
        st.rerun()

# -----
# Configuración de la página
# -----
st.set_page_config(
    page_title="CICORINA",
    page_icon="🌿",
    layout="centered"
)

# -----
# Creación de Pestañas (Tabs)

```

```
# -----
tab_inicio, tab_registro, tab_historial = st.tabs([
    "🏠 Inicio",
    "📝 Registro y Análisis",
    "📊 Historial de Resultados"
])
```

```
# =====
# PESTAÑA 1: INICIO o HOMEPAGE
# =====
```

```
with tab_inicio:
    st.title("🌿 CICORINA")
    st.subheader("Sistema Inteligente para la Clasificación del Nivel de Estrés")
```

```
logo = Path("assets/chikorita.png")
if logo.exists():
    st.image(str(logo), width=250)
```

```
st.write(
    """
    Bienvenido al aplicativo **CICORINA**.
```

Este es un sistema de análisis de estrés que integra evaluación subjetiva y fisiológica en estudiantes universitarios.

A partir de la señal ECG, el sistema realiza una clasificación automáticamente del nivel de estrés de la persona, contrastando la percepción subjetiva de estrés con la reactividad fisiológica objetiva ante el estrés agudo.

```
"""
```

```
)
```

```
imagen_inicio = Path("images (2).jpeg")
if imagen_inicio.exists():
    st.image(str(imagen_inicio), use_container_width=True)
else:
    st.info("💡 (Aquí se mostrará tu imagen de inicio cuando la agregues a la carpeta raíz del proyecto)")
```

```
st.write("---")
```

```
# =====
# PESTAÑA 2: REGISTRO Y RESULTADOS ACTUALES
# =====
```

```
with tab_registro:
    st.title("Carga de datos y Procesamiento")
```

```
st.write("Ingrese el puntaje del cuestionario PSS-14, su información académica y cargue la señal ECG.")
```

```
col_in1, col_in2 = st.columns(2)
```

```
with col_in1:
```

```
    st.subheader("📄 Puntaje del cuestionario PSS-14")
```

```
    puntaje_pss = st.number_input(
```

```
        "Ingrese el puntaje total (0 - 56)",
```

```
        min_value=0,
```

```
        max_value=56,
```

```
        step=1,
```

```
        key=f"input_pss_{st.session_state.form_version}"
```

```
    )
```

```
    st.divider()
```

```
    st.subheader("📖 Información académica")
```

```
    horas_estudio = st.number_input(
```

```
        "Horas dedicadas a la universidad por semana",
```

```
        min_value=0,
```

```
        max_value=120,
```

```
        value=20,
```

```
        step=1,
```

```
        key=f"input_horas_{st.session_state.form_version}"
```

```
    )
```

```
    st.divider()
```

```
with col_in2:
```

```
    st.subheader("👤 Características del Usuario")
```

```
    genero = st.selectbox(
```

```
        "Seleccione su género:",
```

```
        ["Femenino", "Masculino", "Prefiero no especificar"],
```

```
        index=2, # Por defecto seleccionará "Prefiero no especificar"
```

```
        key=f"input_genero_{st.session_state.form_version}"
```

```
    )
```

```
    st.divider()
```

```
    st.subheader("❤️ Señal ECG")
```

```
    archivo_ecg = st.file_uploader(
```

```
        "Ingrese el archivo en formato txt de la señal ECG obtenida (BITalino)",
```

```
        type=["txt"],
```

```
        key=f"uploader_ecg_{st.session_state.form_version}"
```

```
    )
```

```
if archivo_ecg is not None:
```

```
    st.success(f"Archivo cargado: {archivo_ecg.name}")
```

```
if st.button("Procesar análisis", use_container_width=True):
```

```
    with st.spinner("Procesando... Por favor espere mientras se realiza el análisis..."):
```

```
        try:
```

```
            # 1. CARGA DE DATOS
```

```

archivo_ecg.seek(0)
lineas_numericas = []

for linea in archivo_ecg:
    if isinstance(linea, bytes):
        linea_str = linea.decode('utf-8').strip()
    else:
        linea_str = linea.strip()

    if not linea_str or linea_str.startswith('#'):
        continue

    valores = linea_str.split()
    try:
        valores_num = [float(v) for v in valores]
        if valores_num:
            lineas_numericas.append(valores_num)
    except ValueError:
        continue

data = np.array(lineas_numericas)
if data.size == 0:
    raise ValueError("No se encontraron filas numéricas válidas.")

if data.ndim == 1:
    signal = data
else:
    if np.isnan(data[:, -1]).all():
        data = data[:, :-1]
    signal = data[:, -1]

signal = signal[~np.isnan(signal)]

if len(signal) < 30:
    raise ValueError(f"La señal limpia es demasiado corta ({len(signal)} muestras).")

fs = 1000.0

# 2. PROCESAMIENTO Y FILTRADO
nyquist = fs / 2.0
low = 0.5 / nyquist
high = 100 / nyquist

b_band, a_band = butter(4, [low, high], btype='band')
filtered_band_signal = filtfilt(b_band, a_band, signal)

q = 30
b_notch, a_notch = iirnotch(60, q, fs)

```

```
filtered_notch_signal = filtfilt(b_notch, a_notch, filtered_band_signal)
```

3. EXTRACCIÓN DEL COMPLEJO QRS

```
diff_signal = np.diff(filtered_notch_signal, prepend=filtered_notch_signal[0])
```

```
squared_signal = diff_signal**2
```

```
window_length = int(0.12 * fs)
```

```
integrated_signal = np.convolve(squared_signal,  
np.ones(window_length)/window_length, mode='same')
```

```
min_distance = int(0.6 * fs)
```

```
peaks, _ = find_peaks(integrated_signal, distance=min_distance,  
height=np.mean(integrated_signal))
```

```
if len(peaks) < 3:
```

```
    raise ValueError("Se detectaron muy pocos picos R. Asegúrate de que el archivo  
contenga una señal ECG válida.")
```

```
rr_interval_s = np.diff(peaks) / fs
```

```
rr_interval_ms = rr_interval_s * 1000
```

4. CÁLCULO DE PARÁMETROS HRV Y FRECUENCIA CARDÍACA (BPM)

```
hr_instantaneo = 60.0 / rr_interval_s
```

```
mean_hr = np.mean(hr_instantaneo)
```

```
sdnn = np.std(rr_interval_ms)
```

```
rmssd = np.sqrt(np.mean(np.diff(rr_interval_ms)**2))
```

```
time_peaks = peaks[1:] / fs
```

```
interp_func = interp1d(time_peaks, rr_interval_ms, kind='cubic',  
fill_value="extrapolate")
```

```
fs_resample = 4.0
```

```
time_resample = np.arange(time_peaks[0], time_peaks[-1], 1/fs_resample)
```

```
rr_resampled = interp_func(time_resample)
```

```
frequencies, psd = welch(rr_resampled, fs=fs_resample, nperseg=256)
```

```
lf_band = (frequencies >= 0.04) & (frequencies <= 0.15)
```

```
hf_band = (frequencies >= 0.15) & (frequencies <= 0.40)
```

```
lf_power = trapezoid(psd[lf_band], frequencies[lf_band])
```

```
hf_power = trapezoid(psd[hf_band], frequencies[hf_band])
```

```
lf_hf_ratio = lf_power / hf_power if hf_power > 0 else 0
```

```
if puntaje_pss <= 18:
```

```
    clasificacion = "Bajo"
```

```
elif puntaje_pss <= 37:
```

```

        clasificacion = "Moderado"
    else:
        clasificacion = "Alto"

```

5. VISUALIZACIÓN COMPLETA (Generación de gráficos)

```

fig, axs = plt.subplots(3, 1, figsize=(12, 10))
n = np.arange(len(signal))
t = n / fs

```

Subplot 1: Señal Cruda vs Filtrada Completa

```

axs[0].plot(t, signal, label="Cruda (BITalino)", alpha=0.5, color='gray')
axs[0].plot(t, filtered_notch_signal, label="Filtrada (0.5-100 Hz + Notch)", linewidth=1,
color='blue')
axs[0].set_title("Señal de ECG en el Dominio del Tiempo (Sesión Completa)")
axs[0].set_ylabel("Amplitud / Cuentas ADC")
axs[0].legend(loc="upper right")
axs[0].grid(ls="--", alpha=0.5)

```

Subplot 2: Todos los Picos Detectados

```

axs[1].plot(t, integrated_signal, color='green', label="Señal Integrada")
axs[1].plot(peaks / fs, integrated_signal[peaks], 'rx', markersize=7,
markeredgewidth=1.5, label="Todos los Picos R")
axs[1].set_title("Procesamiento QRS y Localización de Latidos (Toda la señal)")
axs[1].set_ylabel("Magnitud")
axs[1].legend(loc="upper right")
axs[1].grid(ls="--", alpha=0.5)

```

Subplot 3: Evolución de la Frecuencia Cardíaca (BPM)

```

axs[2].plot(time_peaks, hr_instantaneo, marker='o', markersize=3, linestyle='-',
color='crimson', mfc='white', alpha=0.8, label="BPM Instantáneo")
axs[2].axhline(y=mean_hr, color='black', linestyle='--', alpha=0.7, label=f'Media:
{mean_hr:.1f} BPM')
axs[2].set_title("Evolución de la Frecuencia Cardíaca (BPM) a lo largo del tiempo")
axs[2].set_xlabel("Tiempo de la sesión (s)")
axs[2].set_ylabel("Frecuencia Cardíaca (BPM)")
axs[2].legend(loc="upper right")
axs[2].grid(ls="--", alpha=0.5)

```

```

plt.tight_layout()

```

CREAR E INYECTAR DICCIONARIO DE RESULTADO

```

nuevo_resultado = {
    "id": len(st.session_state.historial),
    "fecha": datetime.datetime.now().strftime("%d/%m/%Y %H:%M:%S"),
    "pss": puntaje_pss,
    "horas": horas_estudio,
    "genero": genero,
    "clasificacion": clasificacion,

```

```

        "mean_hr": mean_hr,
        "sdnn": sdnn,
        "rmssd": rmssd,
        "lf_hf_ratio": lf_hf_ratio,
        "figura": fig
    }

```

```

# Guardar estructuradamente en session_state
st.session_state.registro_actual = nuevo_resultado
st.session_state.historial.append(nuevo_resultado)

```

```

st.success("✅ Análisis completado con éxito.")

```

```

except Exception as e:

```

```

    st.error(f"Ocurrió un error al procesar el archivo: {e}")

```

```

# VER RESULTADOS EN LA MISMA PESTAÑA DE REGISTRO

```

```

if st.session_state.registro_actual is not None:

```

```

    st.divider()

```

```

    st.markdown("## 📌 Resultado del Análisis Reciente")

```

```

    mostrar_bloque_resultados(st.session_state.registro_actual)

```

```

# -----

```

```

# REGISTRAR NUEVOS VALORES

```

```

# -----

```

```

st.write("") # Espacio estético

```

```

if st.button("🔄 Registrar nuevos valores", use_container_width=True):

```

```

    st.session_state.registro_actual = None

```

```

    st.session_state.form_version += 1

```

```

    st.rerun()

```

```

# =====

```

```

# PESTAÑA 3: HISTORIAL

```

```

# =====

```

```

with tab_historial:

```

```

    if st.session_state.vista_detalle_id is None:

```

```

        st.title("📖 Historial de Análisis")

```

```

        st.write("Selecciona un análisis guardado para abrir su reporte detallado y gráficas.")

```

```

        st.divider()

```

```

    if not st.session_state.historial:

```

```

        st.info("Aún no se han procesado registros en esta sesión.")

```

```

    else:

```

```

        for item in reversed(st.session_state.historial): # El más reciente primero

```

```

            col_h1, col_h2 = st.columns([3, 1])

```

```

            with col_h1:

```

```

        st.markdown(f" **Fecha:** `{item['fecha']}` |  **Estrés Subjetivo:**  

`{item['clasificacion']}`")
        with col_h2:
            if st.button("Ver Detalle", key=f"btn_hist_{item['id']}", use_container_width=True):
                st.session_state.vista_detalle_id = item['id']
                st.rerun()
            st.write("---")

    else:
        if st.button(" Volver al Historial Completo", use_container_width=True):
            st.session_state.vista_detalle_id = None
            st.rerun()

    st.divider()

    detalle = next(item for item in st.session_state.historial if item["id"] ==
st.session_state.vista_detalle_id)
    mostrar_bloque_resultados(detalle)

```